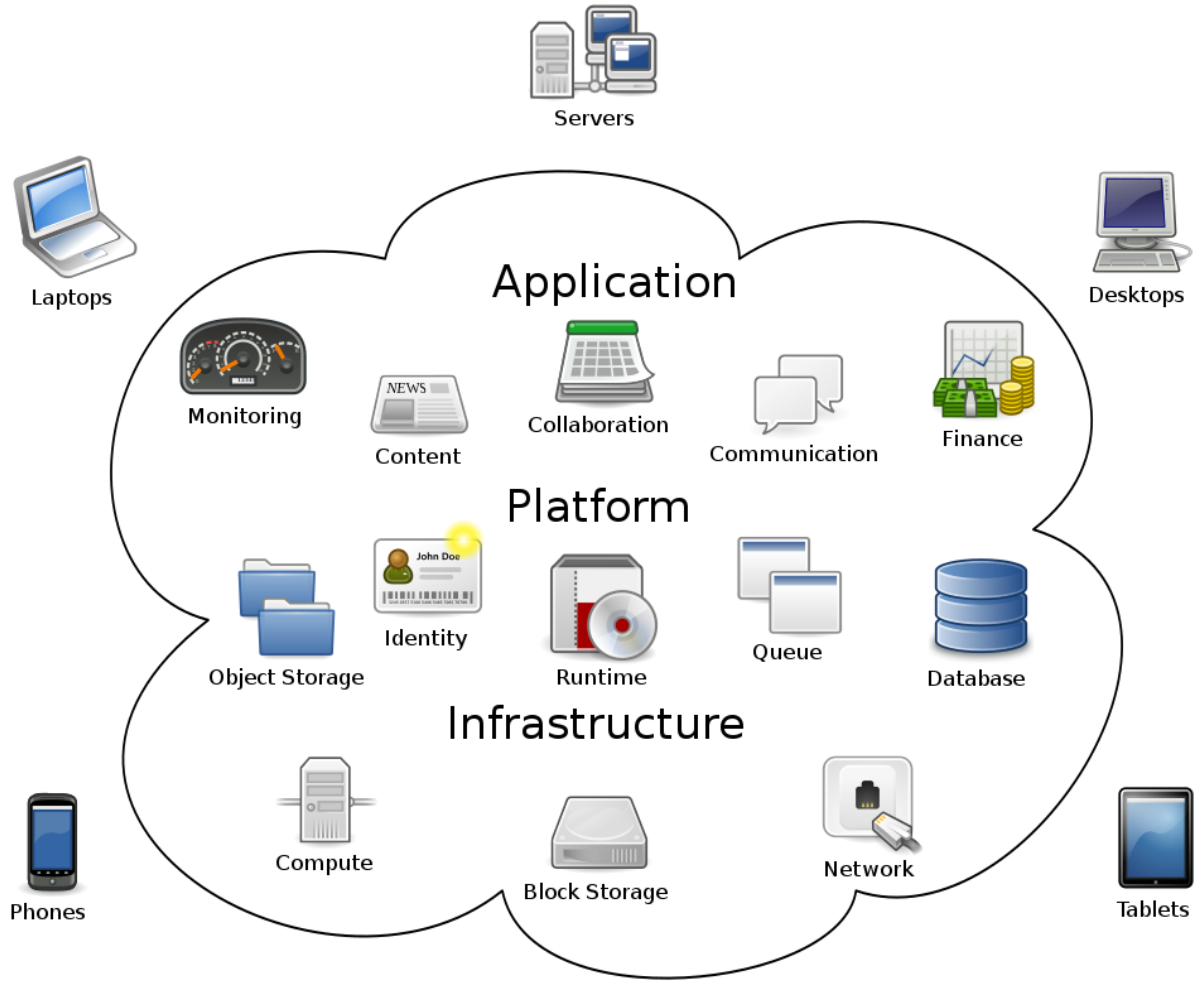
# VPC Introduction

Course: Networking Principles in Practice – Cloud Networking
Module: VPC



University of Colorado **Boulder**

# How the cloud is organized



# Datacenter

**Nederlands**



**Inside a Google data center**
https://www.youtube.com/watch?v=XZmGGAbHqa0&ab_channel=GoogleWorkspace

# Regions

- Correlate to locations of Google Datacenters
- Named based on location (e.g., us-west1, asia-east1)
- gcloud compute regions list




https://cloud.google.com/about/locations#lightbox-regions-map

# Zones

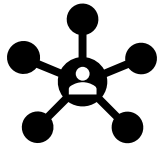
- Critical shared resources for compute infrastructure:



power

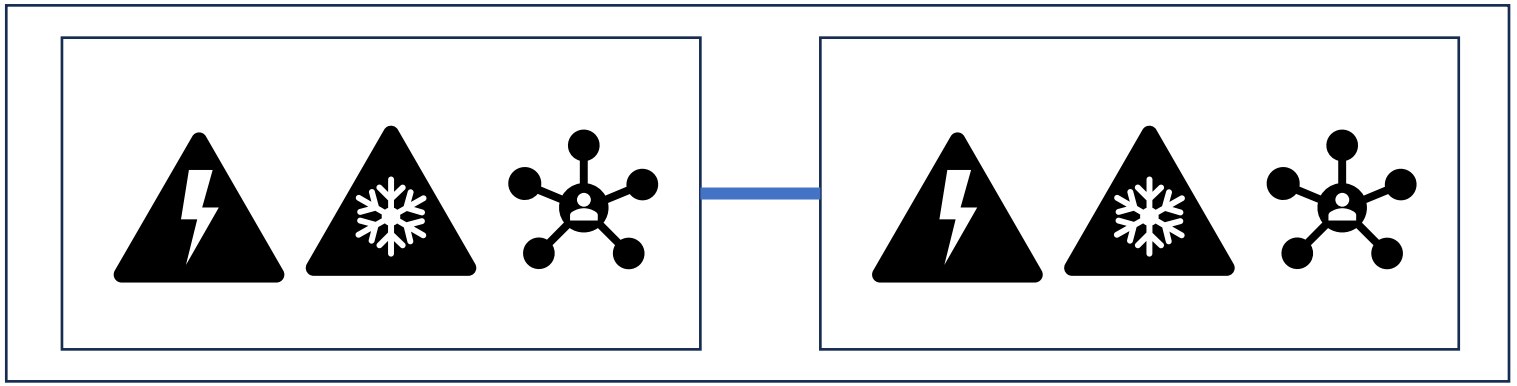


cooling



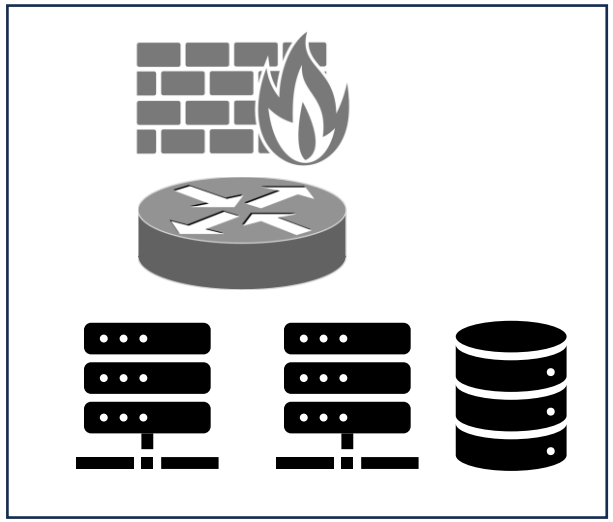
Networking

- Zones provide Fault Isolation within a region, interconnected with fiber



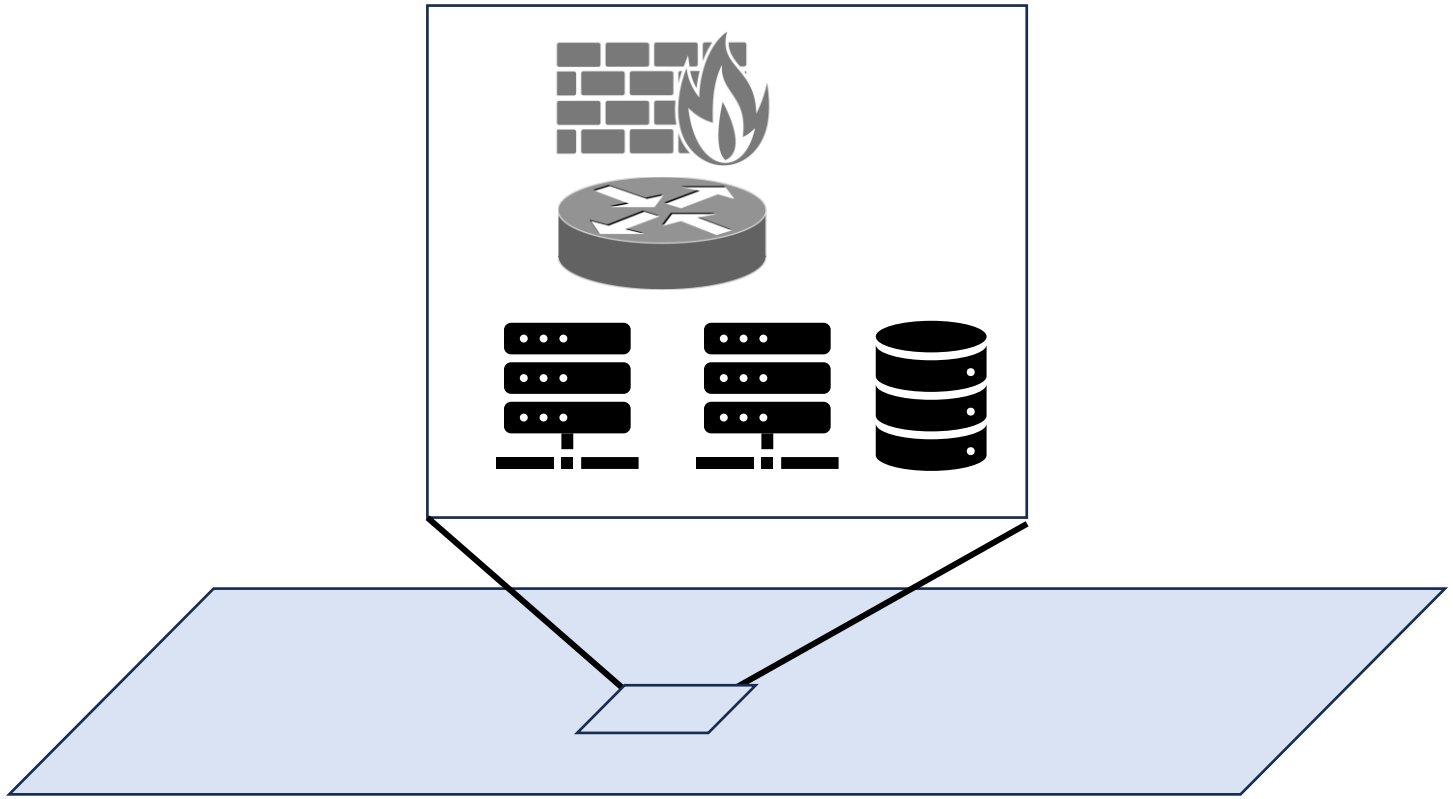
# Virtual Private Cloud (VPC)

- Private Cloud



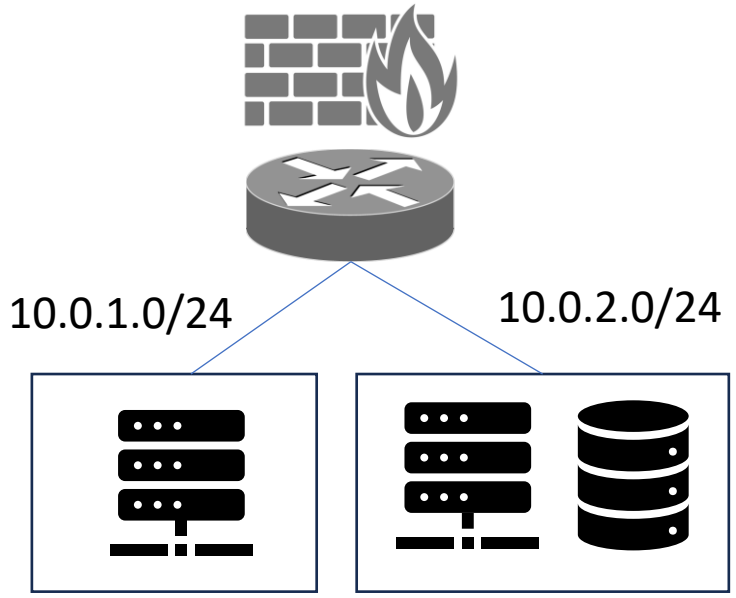
# Virtual Private Cloud (VPC)

- Virtual Private Cloud – effectively your own private cloud running on top of the providers physical infrastructure



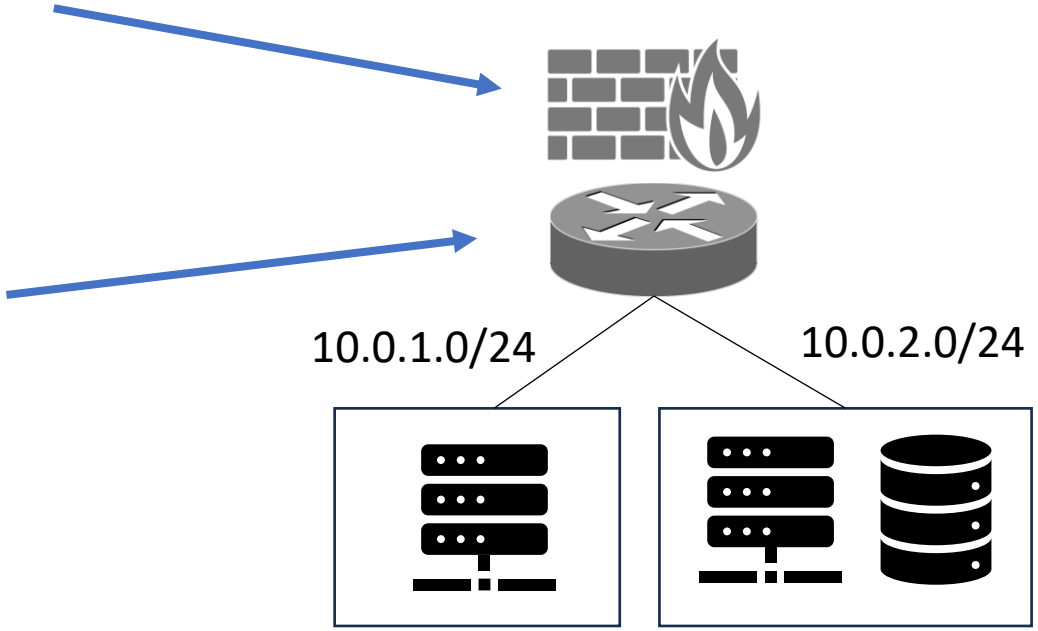
# Subnets

- Subnet – Subdivision of a network through partitioning IP address space into subsets



# Routes / Rules

- Routes – to direct traffic between diff. subnets and external gateways
- Rules – firewall policy configuration for what traffic to block/allow



# Module Overview

- Creating a VPC in GCP
- Software Defined Networking
- Google's Andromeda
- Azure's AccelNet (SmartNIC)
- VPC – a bit deeper
  (and programming assignment overview)



University of Colorado **Boulder**

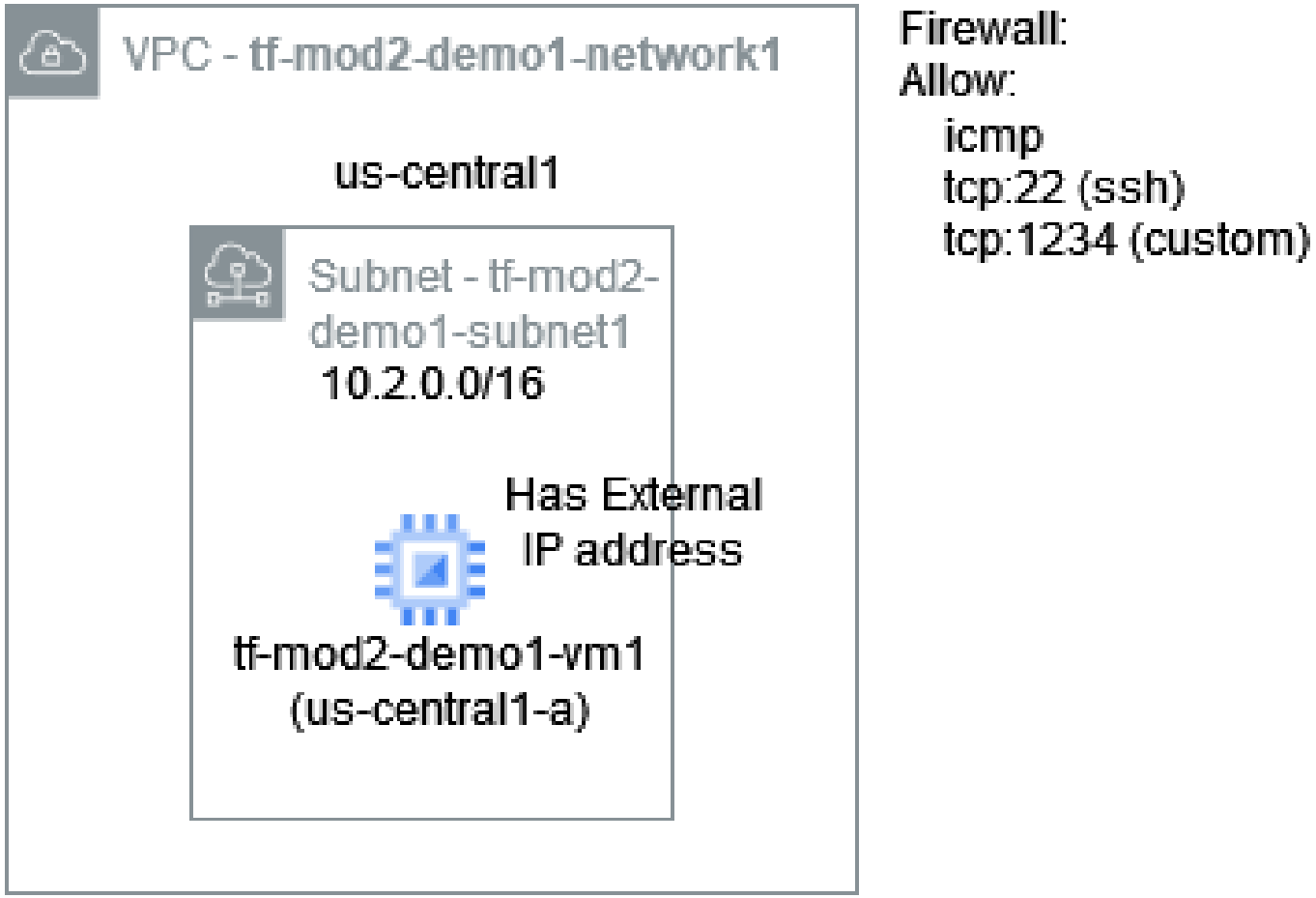
# Creating a VPC in GCP

Course: Networking Principles in Practice – Cloud Networking
Module: VPC



University of Colorado **Boulder**

# Diagram




Firewall:
Allow:
- icmp
- tcp:22 (ssh)
- tcp:1234 (custom)

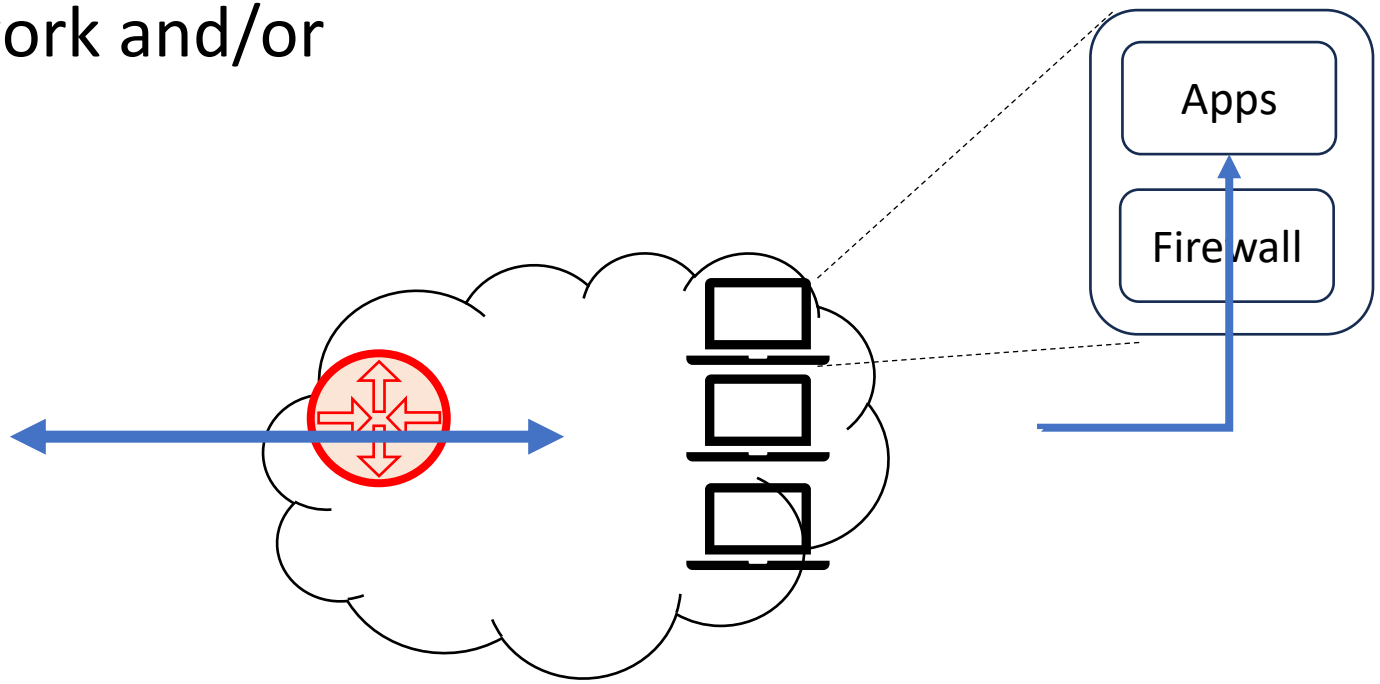
# Walkthrough in GCP Console

- Creating VPC with a subnet
- Create a VM

# Adding Firewall Rules in GCP

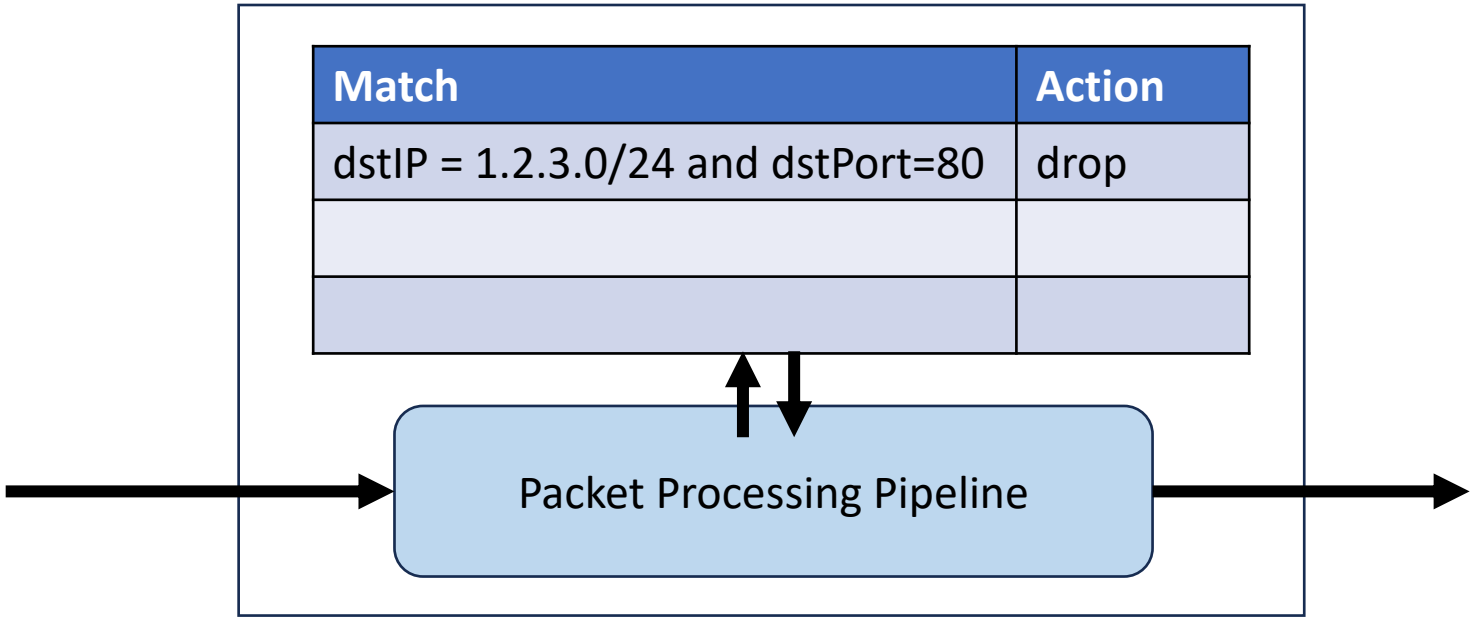
# Filtering

- Want to block/allow traffic according to some policy
  - Web server: block all traffic not for port 80
  - Home gateway: block all traffic except for connections initiated from internal
- Can be at edge of some network and/or
  On the end hosts themselves



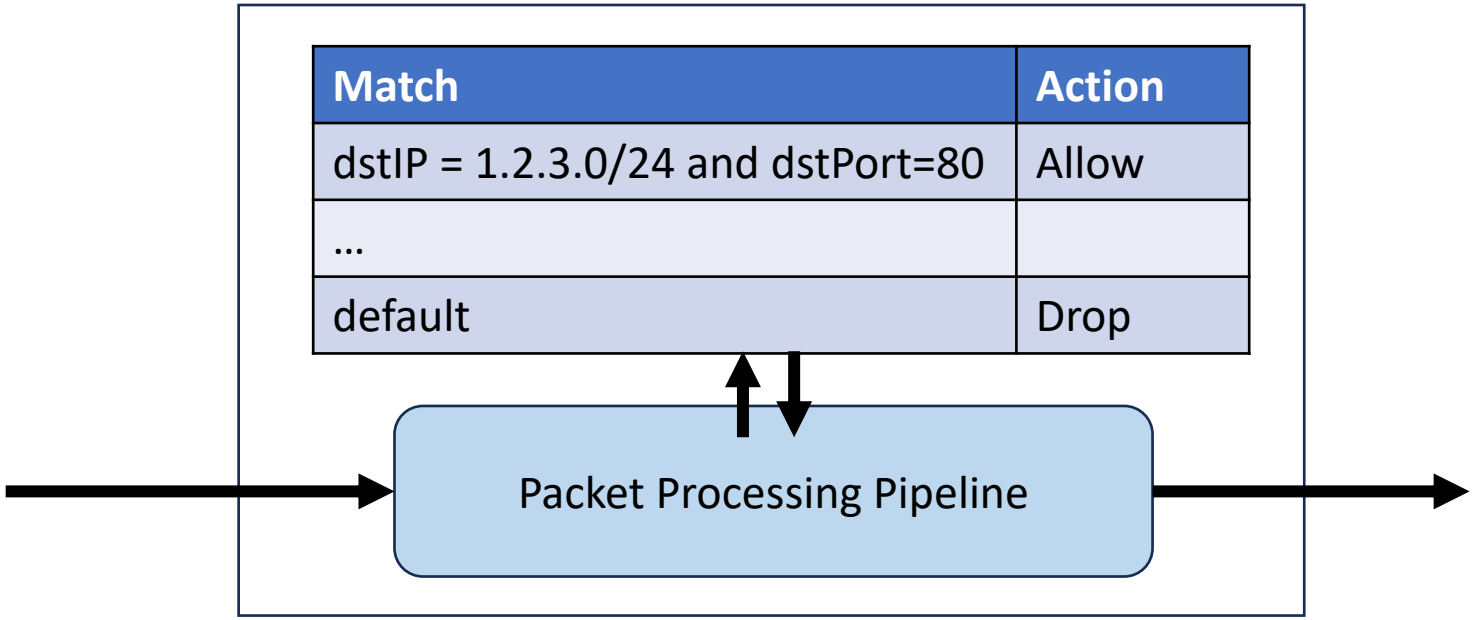
# Match-Actions Tables

- Match - compare input packets to the rule to see if there's a match
  - e.g., dstIP = 1.2.3.0/24 and dstPort=80
- Action – what to do with the packet if there is a match
  - E.g., drop, allow



# Good Practice – Default Drop

- Set the default action to drop
- Add rules to have an action other than drop



# Adding Firewall Rules in GCP

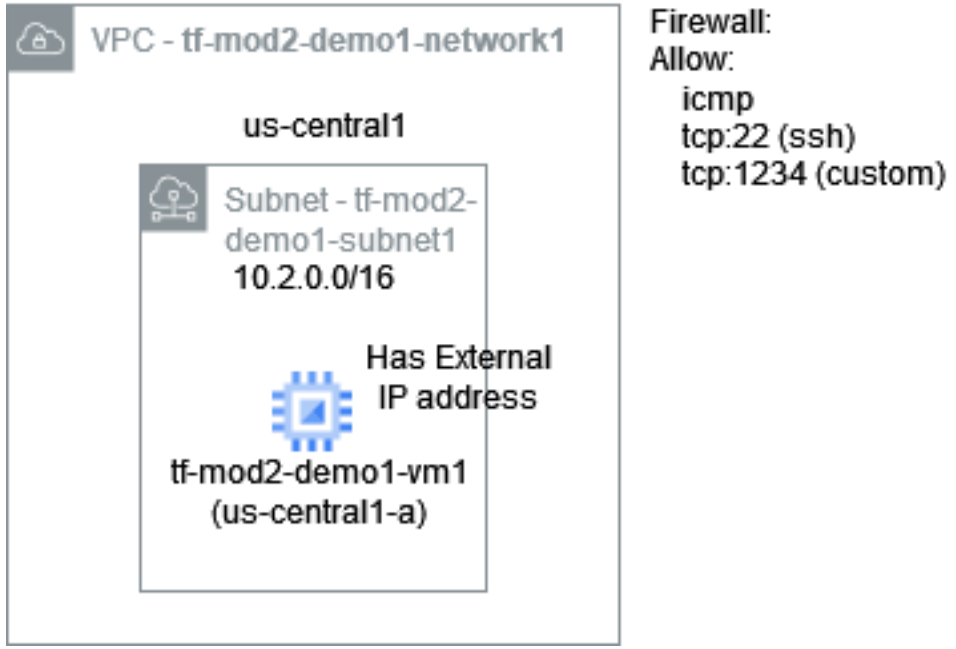
- Two ways to get to it
  - Firewall (will show all rules for all VPCs)
  - VPC Networks -> click on the VPC -> click on Firewall tab
- (Walkthrough)

# Terraform

- VPC - https://registry.terraform.io/providers/hashicorp/google/latest/docs/resources/compute_network
- Subnet - https://registry.terraform.io/providers/hashicorp/google/latest/docs/resources/compute_subnetwork
- VM - https://registry.terraform.io/providers/hashicorp/google/latest/docs/resources/compute_instance
- Firewall - https://registry.terraform.io/providers/hashicorp/google/latest/docs/resources/compute_firewall

# Walkthrough terraform

- terraform init
- terraform plan
- terraform apply
- show in GCP Console
- terraform destroy




https://github.com/eric-keller/npp-cloud/tree/main/tf-mod2-demo1

# Testing

# netcat

- simple unix utility which reads and writes data across network connections, using TCP or UDP protocol
  https://www.commandlinux.com/man-page/man1/nc.1.html

server> nc -l -p 1234  // listen on port 1234

client> echo “blah” | nc 1.2.3.4 1234  // connect to port 1234

# socat – Socket cat

- socat src,opt1,opt2  dst,opt1,opt2
- Src:
  - - (for std in)
  - tcp-listen:port
  - openssl-listen:port
- Dst
  - - (for std out)
  - tcp4, tcp6 – connect to TCP end point

https://linux.die.net/man/1/socat

# Example

socat – tcp4:1.2.3.4:8080

Run on source – everything received on std in (src), will be sent to node 2 port 8080 via tcp (dst)

socat tcp4-listen:8080 –

Run on sink – everything received on tcp port 8080 (src) will be sent to std out (dst)

# Other examples

- As a proxy
  A (socat – tcp:1.2.3.4:1234) ->
  B (socat tcp-listen tcp:1234) ->
  C (tcp-listen:1234 -)
- As an echo server:
  A (socat – tcp,1.2.3.4) ->
  B (socat -v tcp-l:1234,fork exec:'/bin/cat')

# Demo

- Show socat on VM in GCP, socat on laptop
  - Try different ports
- Be sure to run terraform destroy



University of Colorado **Boulder**

# Software Defined Networking

Course: Networking Principles in Practice – Cloud Networking
Module: VPC

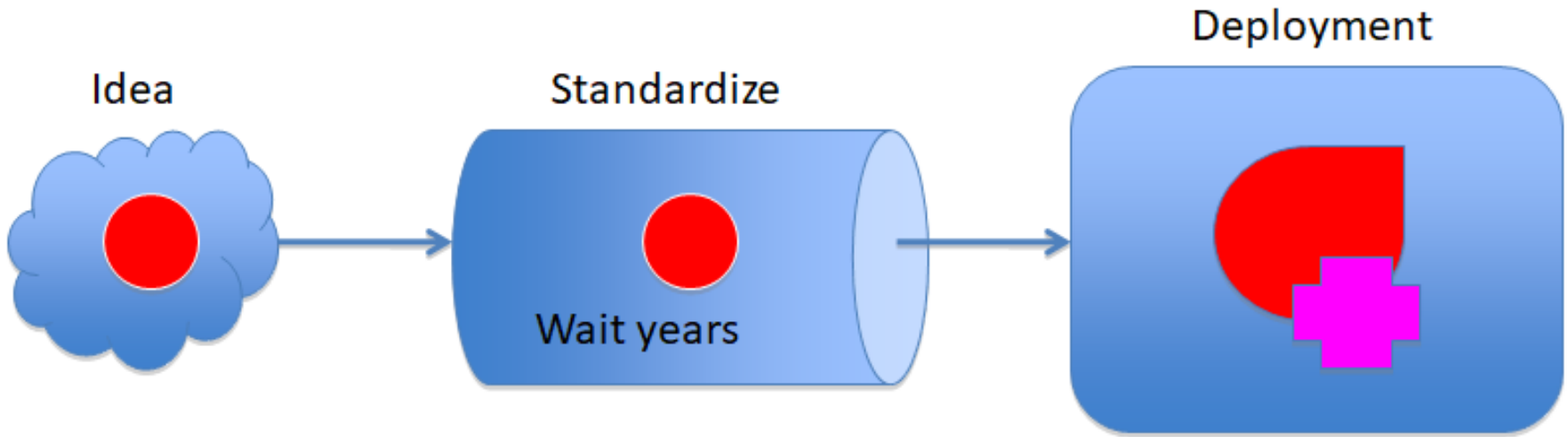


University of Colorado **Boulder**

# History

- What was the feeling when SDN emerged?

# Glacial process of innovation made worse by captive standards process




- Driven by vendors
- Consumers largely locked out
- Lowest common denominator features
- Glacial innovation

# Built an Artifact, Not a Discipline

- Other fields in “systems”: OS, DB, DS, etc.
  - Teach basic principles
  - Are easily managed
  - Continue to evolve
- Networking:
  - Teach big bag of protocols
  - Notoriously difficult to manage
  - Evolves very slowly

# Why Does Networking Lag Behind?

- Networks used to be simple: Ethernet, IP, TCP….

- New **control** requirements led to great complexity
  - Isolation ➔ VLANs, ACLs
  - Traffic engineering ➔ MPLS, ECMP, Weights
  - Packet processing middleboxes ➔ Firewalls, NATs,
  - Payload analysis ➔ Deep packet inspection (DPI)
  - …..

- Mechanisms designed and deployed independently

# Infrastructure Still Works!

- **Only** because of “our” ability to master complexity

- This ability to master complexity is both a blessing…
  - …and a curse!

# Complexity vs Simplicity

- Networking still focused on mastering complexity
  - Little emphasis on extracting simplicity from control plane
  - No recognition that there's a difference….
- Extracting simplicity builds intellectual foundations
  - **Necessary for creating a discipline….**
  - **That's why networking lags behind**

# Example - OSPF (find shortest path)

- RFC 2328: **245 pages**

Distributed System (no abstractions, so do it all)

- Builds consistent, up-to-date map of the network: **101 pages**

Dijkstra's Algorithm

- Operates on map: **4 pages**

# A Better Example: Programming

- Machine languages: no abstractions
  - Mastering complexity was crucial
- Higher-level languages: OS and other abstractions
  - File system, virtual memory, abstract data types, ...
- Modern languages: even more abstractions
  - Object orientation, garbage collection,...

**Abstractions key to extracting simplicity**

# Stepping back - Legacy Switches



Config interface

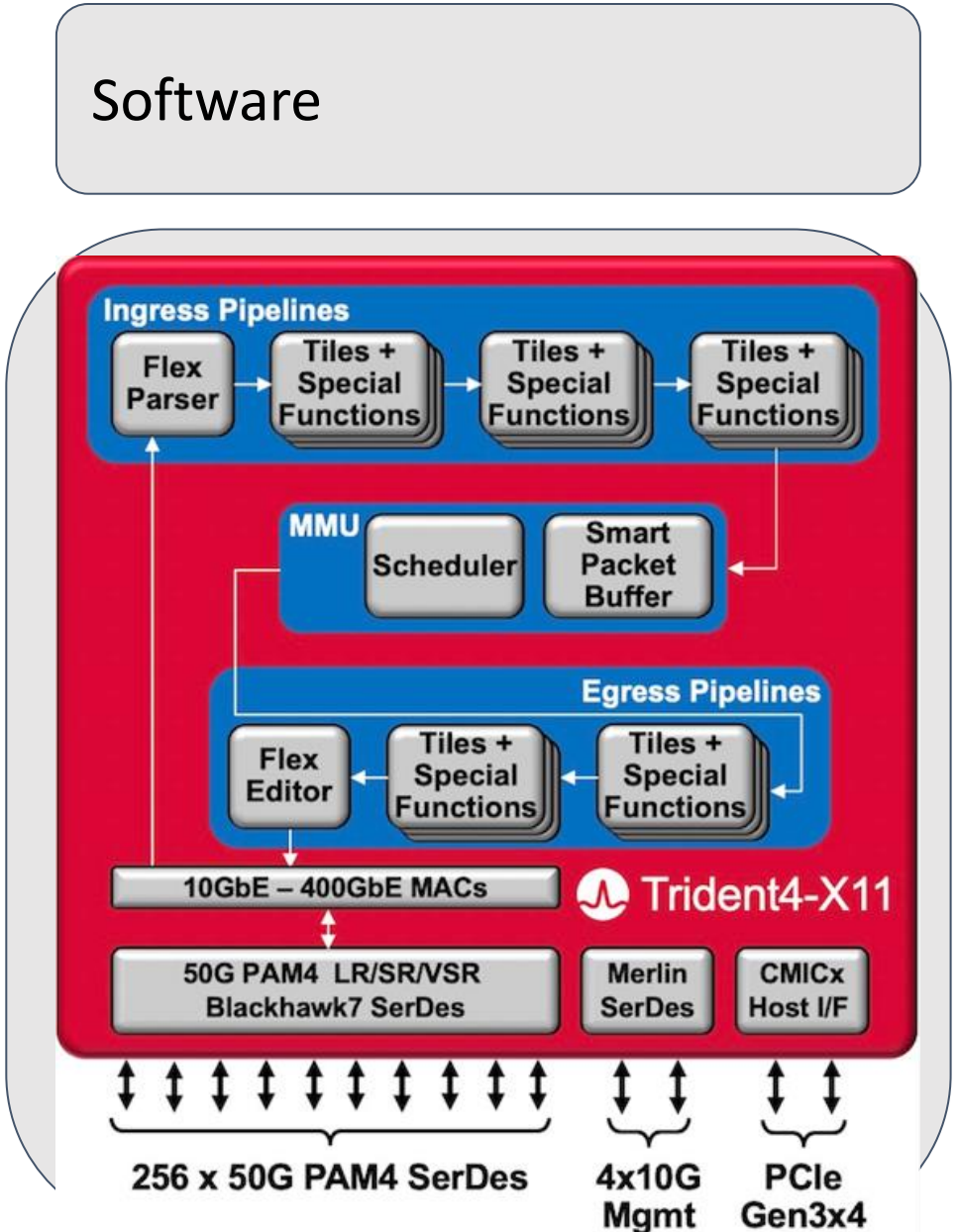
Software

Hardware

# Stepping back - Legacy Switches



Underlying ASIC - Fixed pipeline of fixed functionality

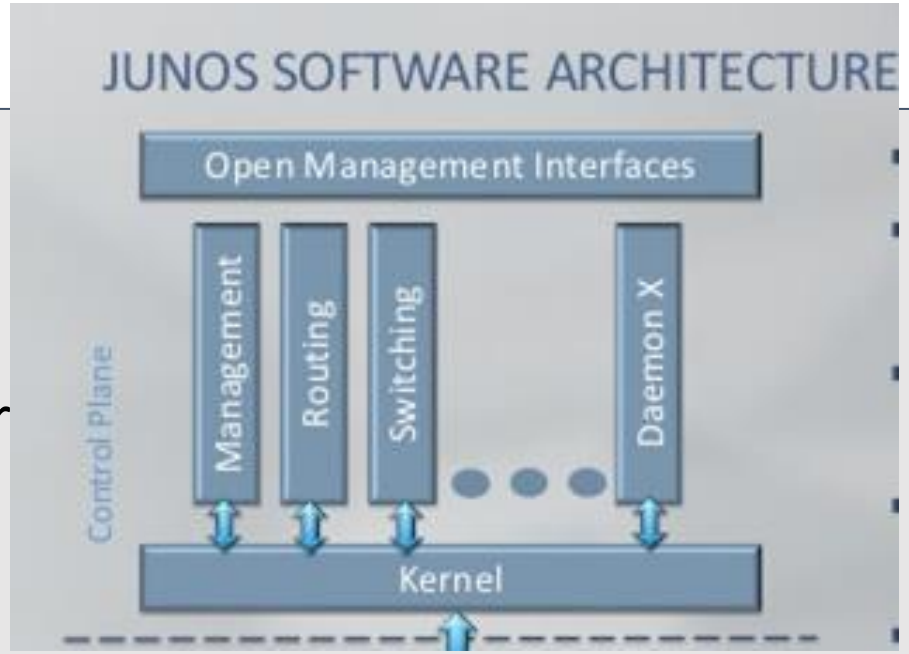


# Stepping back - Legacy Switches

Packaged with routing processes, and ways to configure them.

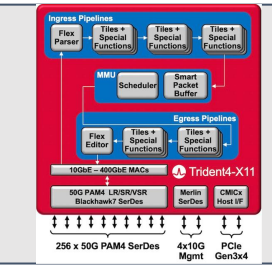


Softwar




Netconf, CLI

Hardware



# Paper in CCR 2005

## A Clean Slate 4D Approach to Network Control and Management ·

Albert Greenberg, Gisli Hjalmtysson, David A. Maltz, Andy Myers,
Jennifer Rexford, Geoffrey Xie, Hong Yan, Jibin Zhan, Hui Zhang
{dmaltz,acm,yh,jibin,hzhang}@cs.cmu.edu
gisli@ru.is jrex@cs.princeton.edu albert@research.att.com xie@nps.edu

### ABSTRACT

Today's data networks are surprisingly fragile and difficult to manage. We argue that the root of these problems lies in the complexity of the control and management planes—the software and protocols coordinating network elements—and particularly the way the decision logic and the distributed-systems issues are inexorably intertwined. We advocate a complete refactoring of the functionality and propose three key principles—network-level objectives, network-wide views, and direct control—that we believe should

### General Terms

Measurement, Control, Performance, Reliability

### Keywords

Network management, robustness, control

### 1. INTRODUCTION

# Problems with the legacy approach

Routers Make Uncoordinated Changes to their Routes

Leads to poor stability and thrashing

**Need a network-wide view, which is needed for a network-level objective**

Must predict and undo effect of control plane

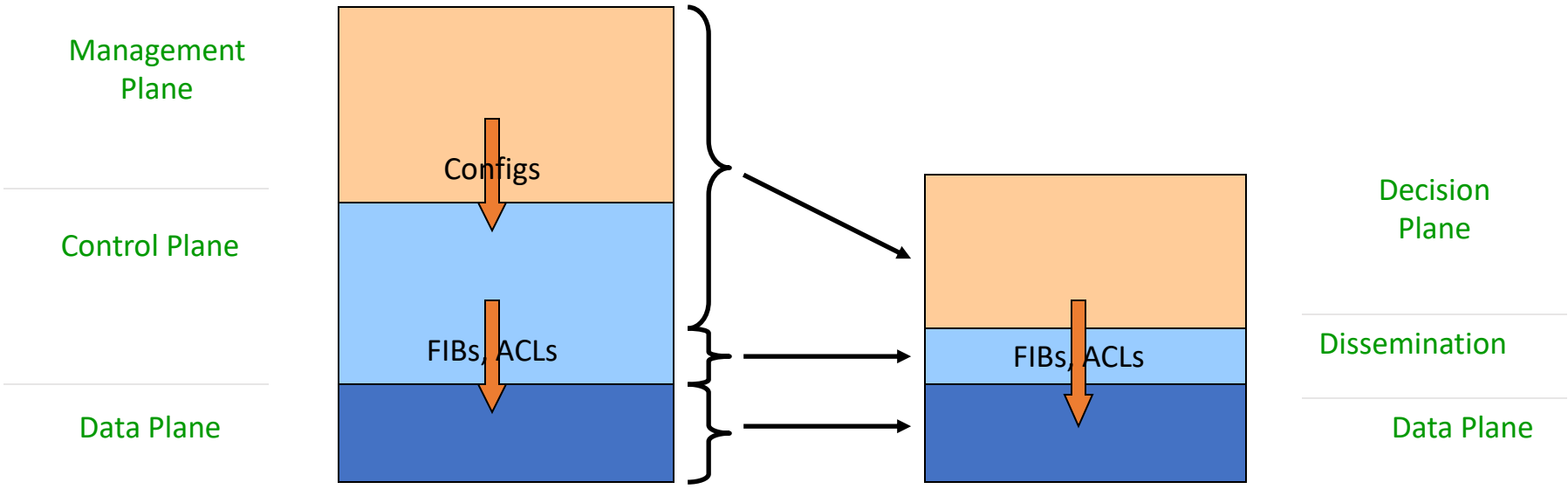
Must translate solution into settings of control plane knobs

**Need ability to express desired solution**

Switch software gets in the way of automation and network-wide network management (need to trick protocols into doing what you need, on switch by switch basis)

# 4D's

- (discovery not shown)



# SDN - (2007) - Ethane

## Ethane: Taking Control of the Enterprise

Martìn Casado, Michael J. Freedman,
Justin Pettit, Jianying Luo,
and Nick McKeown
Stanford University

Scott Shenker
U.C. Berkeley and ICSI

**ABSTRACT**

This paper presents Ethane, a new network architecture for the enterprise. Ethane allows managers to define a single network-wide fine-grain policy, and then enforces it directly. Ethane couples extremely simple flow-based Ethernet switches with a centralized controller that manages the admittance and routing of flows. While radical, this design is backwards-compatible with existing hosts and switches.

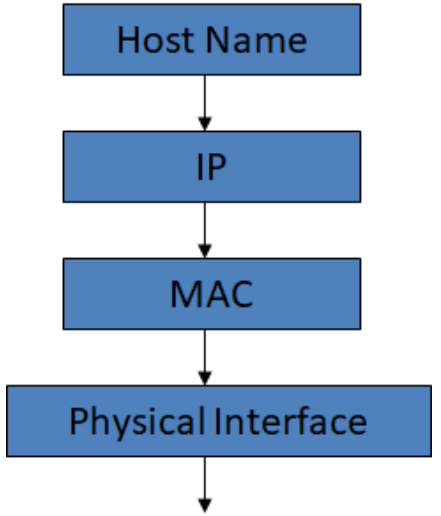
We have implemented Ethane in both hardware and software, supporting both wired and wireless hosts. Our operational Ethane network has supported over 300 hosts for the past four months in in Stanford University's network, and this deployment experience has significantly affected Ethane's design.

downtime in multi-vendor networks comes from human-error and that 80% of IT budgets is spent on maintenance and operations [16].

There have been many attempts to make networks more manageable and more secure. One approach introduces proprietary middleboxes that can exert their control effectively only if placed at network choke-points. If traffic accidentally flows (or is maliciously diverted) around the middlebox, the network is no longer managed nor secure [25]. Another approach is to add functionality to existing networks—to provide tools for diagnosis, to offer controls for VLANs, access-control lists, and filters to isolate users, to instrument the routing and spanning tree algorithms to support better connectivity management, and then to collect packet traces to allow auditing. This can be done by adding a new layer of protocols, scripts, and applications [1, 10] that help automate configuration

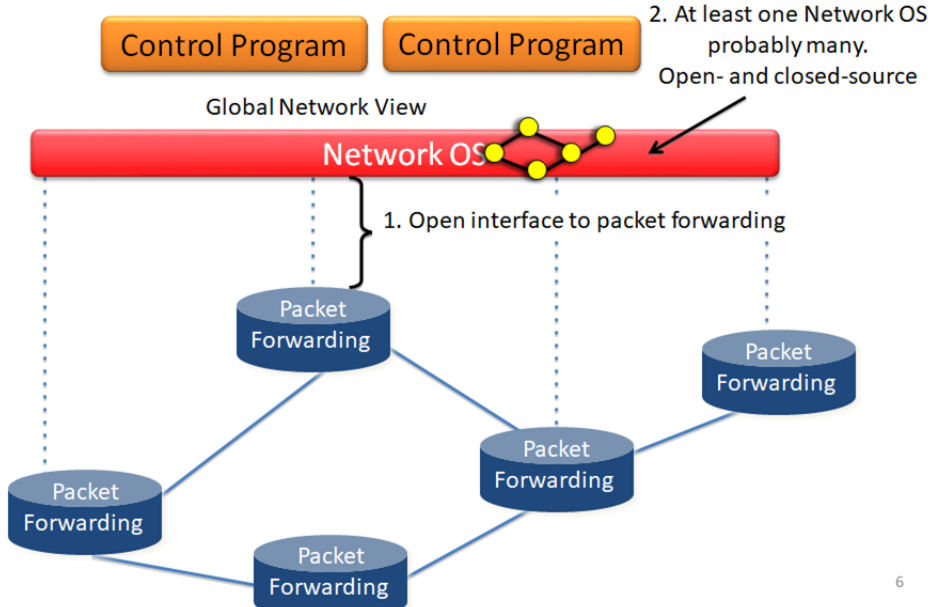
# Ethane's Contribution - Put 4D into Practice (This became OpenFlow)

- Mechanisms for finding and mapping namespaces (Discovery)
  - E.g., you plug a server into a switch, how does centralized controller find out about it




- Interface to describe policy (Decision)
  - "Nick can talk to Martin over IM"

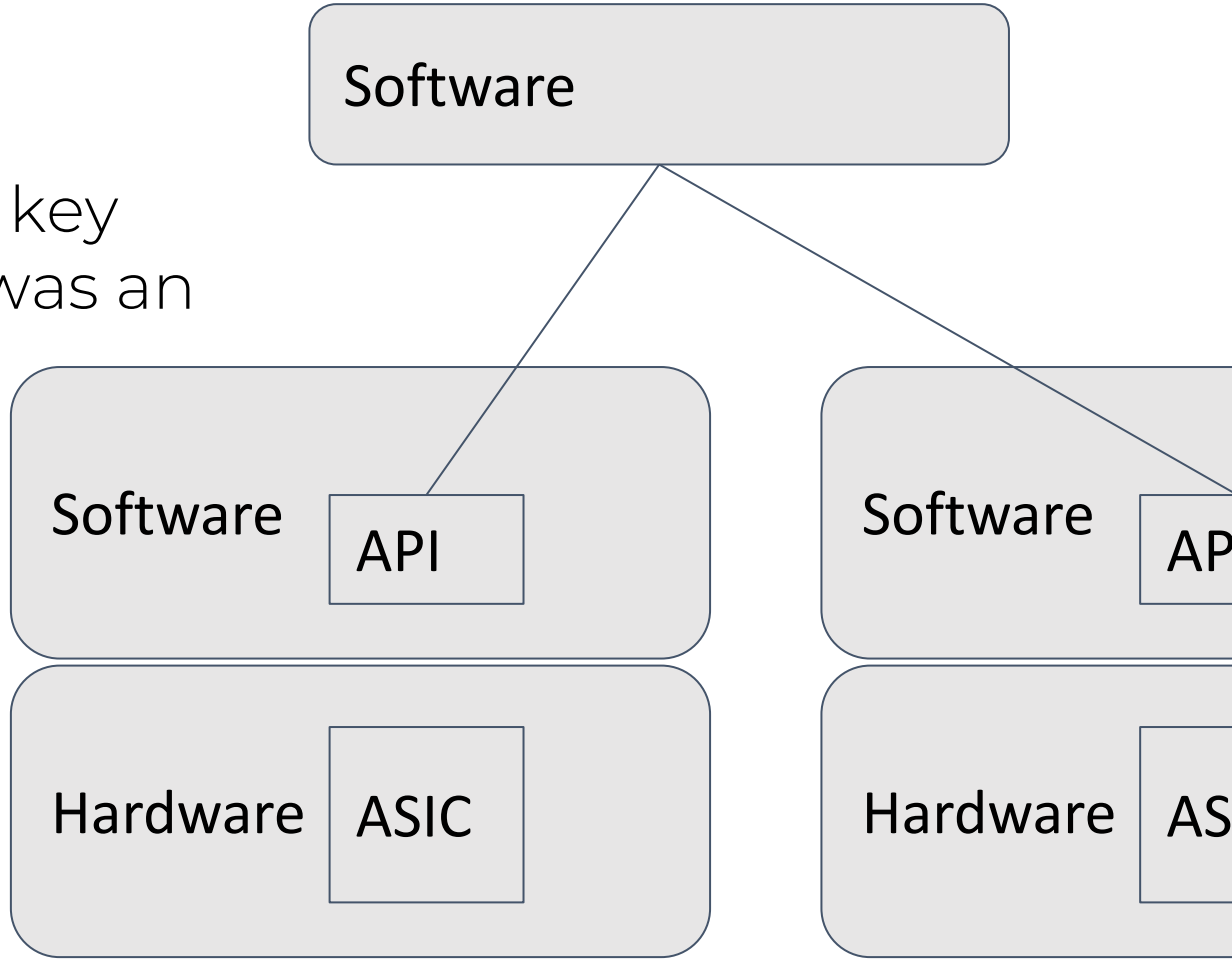
- Mechanism to enforce policy (Data Plane and Dissemination)
  - Match / Action tables in data plane
  - Protocol to write tables



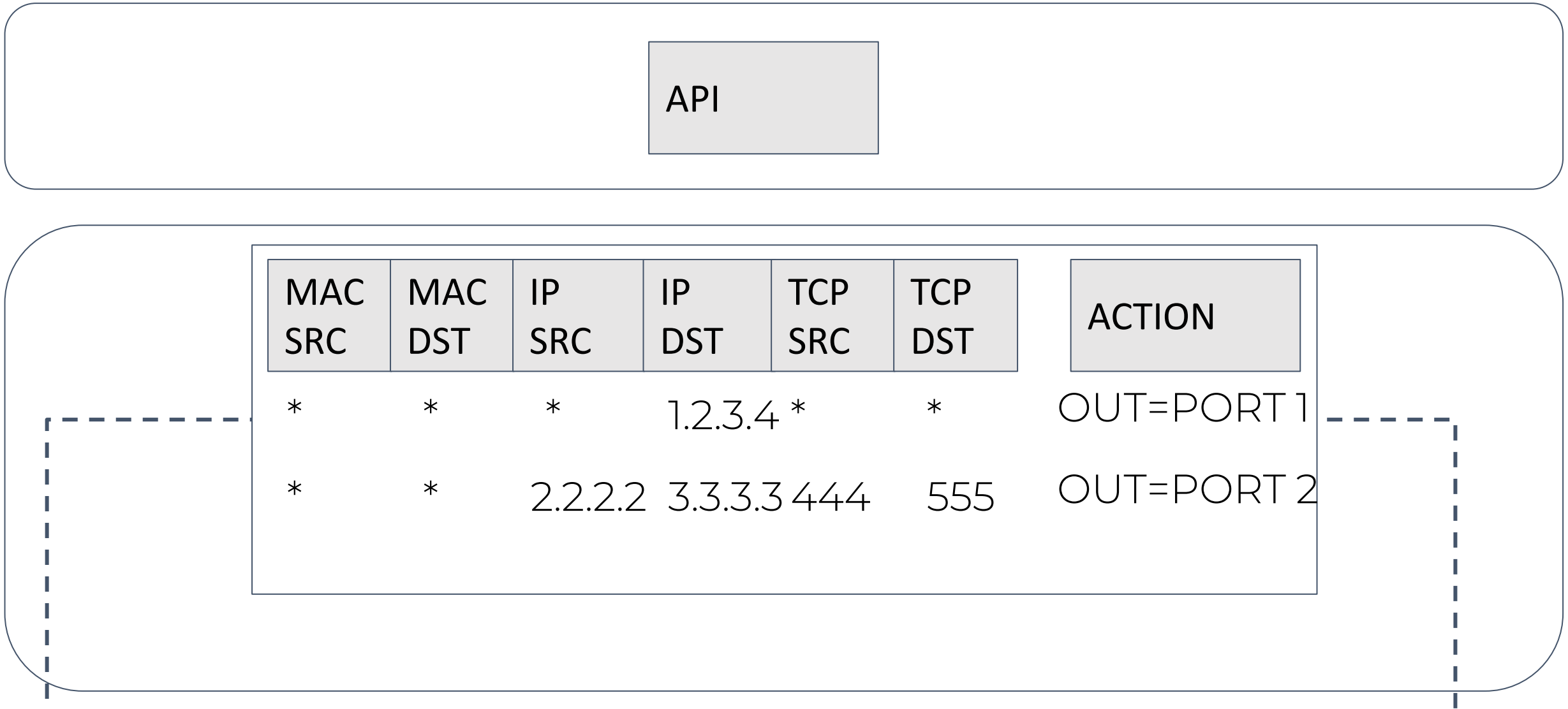
# OpenFlow



OpenFlow - key innovation was an abstraction



# Key Innovation: Match Action Table Abstraction



# *match+action:* unifies different kinds of devices

- Router
  - match: longest destination IP prefix
  - action: forward out a link
- Switch
  - match: destination MAC address
  - action: forward or flood
- Firewall
  - match: IP addresses and TCP/UDP port numbers
  - action: permit or deny
- NAT
  - match: IP address and port
  - action: rewrite address and port

# The Problem with OpenFlow

## Fixed Set of things you can Match on:

Ingress port, Metadata, Ether src, Ether dest, Ether type, VLAN id, VLAN priority, MPLS label, MPLS traffic class, IPv4 src, IPv4 dest, IPV4 proto / ARP opcode, IPv4 ToS bit, TCP/UDP src port, TCP/UDP dest port, ICMP type, ICMP code

## Fixed Set of Actions:

1. Forward packet to port(s)
2.Encapsulate and forward to controller
3.Drop packet
4.Send to normal processing pipeline
5.Modify Fields

**All Dictated by the Fixed ASIC**

The OpenFlow team kept adding protocols to OpenFlow spec, but the HW didn't keep up
(it's not extensible)

# Over Five Years…

Proliferation of header fields

| Version | Date | # Headers |
|---|---|---|
| OF 1.0 | Dec 2009 | 12 |
| OF 1.1 | Feb 2011 | 15 |
| OF 1.2 | Dec 2011 | 36 |
| OF 1.3 | Jun 2012 | 40 |
| OF 1.4 | Oct 2013 | 41 |

Multiple stages of heterogeneous tables

Still not enough (e.g., VXLAN, NVGRE, STT, …)

# Reconfigurable Match Action Tables (RMT)

## Forwarding Metamorphosis [SIGCOMM 2013]

**Forwarding Metamorphosis: Fast Programmable Match-Action Processing in Hardware for SDN**

Pat Bosshart†, Glen Gibb‡, Hun-Seok Kim†, George Varghese§, Nick McKeown‡,
Martin Izzard†, Fernando Mujica†, Mark Horowitz‡
†Texas Instruments ‡Stanford University §Microsoft Research
pat.bosshart@gmail.com {grg, nickm, horowitz}@stanford.edu
varghese@microsoft.com {hkim, izzard, fmujica}@ti.com

### ABSTRACT

In Software Defined Networking (SDN) the control plane is physically separate from the forwarding plane. Control software programs the forwarding plane (e.g., switches and routers) using an open interface, such as OpenFlow. This paper aims to overcomes two limitations in current switching chips and the OpenFlow protocol: i) current hardware switches are quite rigid, allowing "Match-Action" processing on only a fixed set of fields, and ii) the OpenFlow specification only defines a limited repertoire of packet processing actions. We propose the RMT (reconfigurable match tables) model, a new RISC-inspired pipelined architecture for switching chips, and we identify the essential minimal set of action primitives to specify how headers are processed in hardware. RMT allows the forwarding plane to be changed in the field without modifying hardware. As in OpenFlow, the programmer can specify multiple match tables of arbitrary width and depth, subject only to an overall resource limit, with each table configurable for matching on arbitrary fields. However, RMT allows the programmer to modify *all* header fields much more comprehensively than in OpenFlow. Our paper describes the design of a 64 port by 10 Gb/s switch chip implementing the RMT model. Our concrete design demonstrates, contrary to concerns within the community, that flexible OpenFlow hardware switch implementations are feasible at almost no additional cost or power.

### Categories and Subject Descriptors

C.2.1 [**Computer-Communication Networks**]: Network Architecture and Design—*Network Communications*

### 1. INTRODUCTION

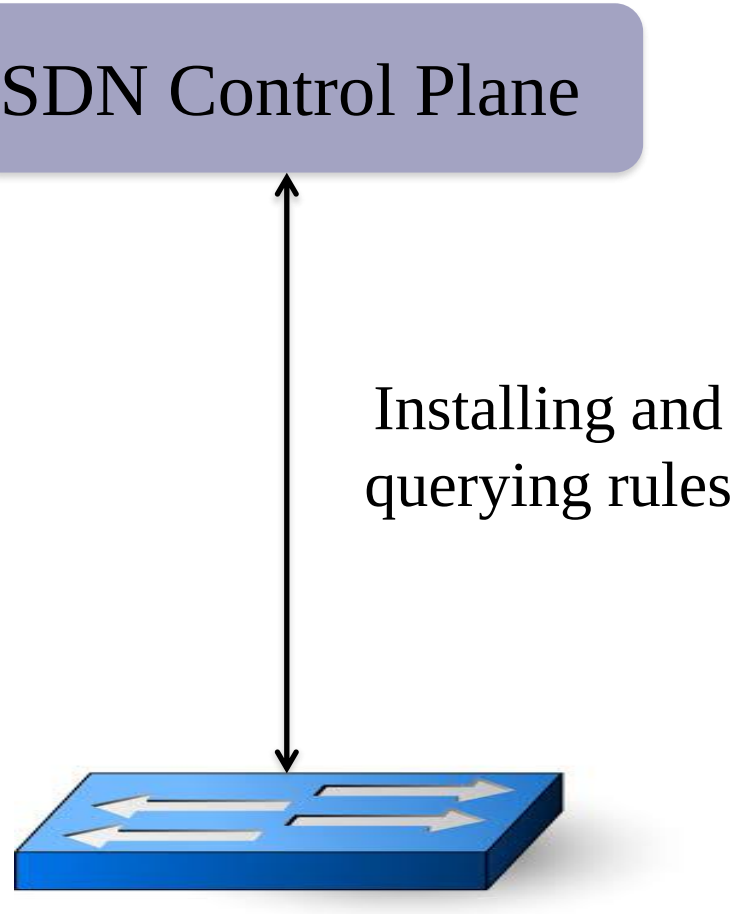
*To improve is to change; to be perfect is to change often.* — Churchill

Good abstractions—such as virtual memory and time-sharing—are paramount in computer systems because they allow systems to deal with change and allow simplicity of programming at the next higher layer. Networking has progressed because of key abstractions: TCP provides the abstraction of connected queues between endpoints, and IP provides a simple datagram abstraction from an endpoint to the network edge. However, routing and forwarding *within* the network remain a confusing conglomerate of routing protocols (e.g., BGP, ICMP, MPLS) and forwarding behaviors (e.g., routers, bridges, firewalls), and the control and forwarding planes remain intertwined inside closed, vertically integrated boxes.

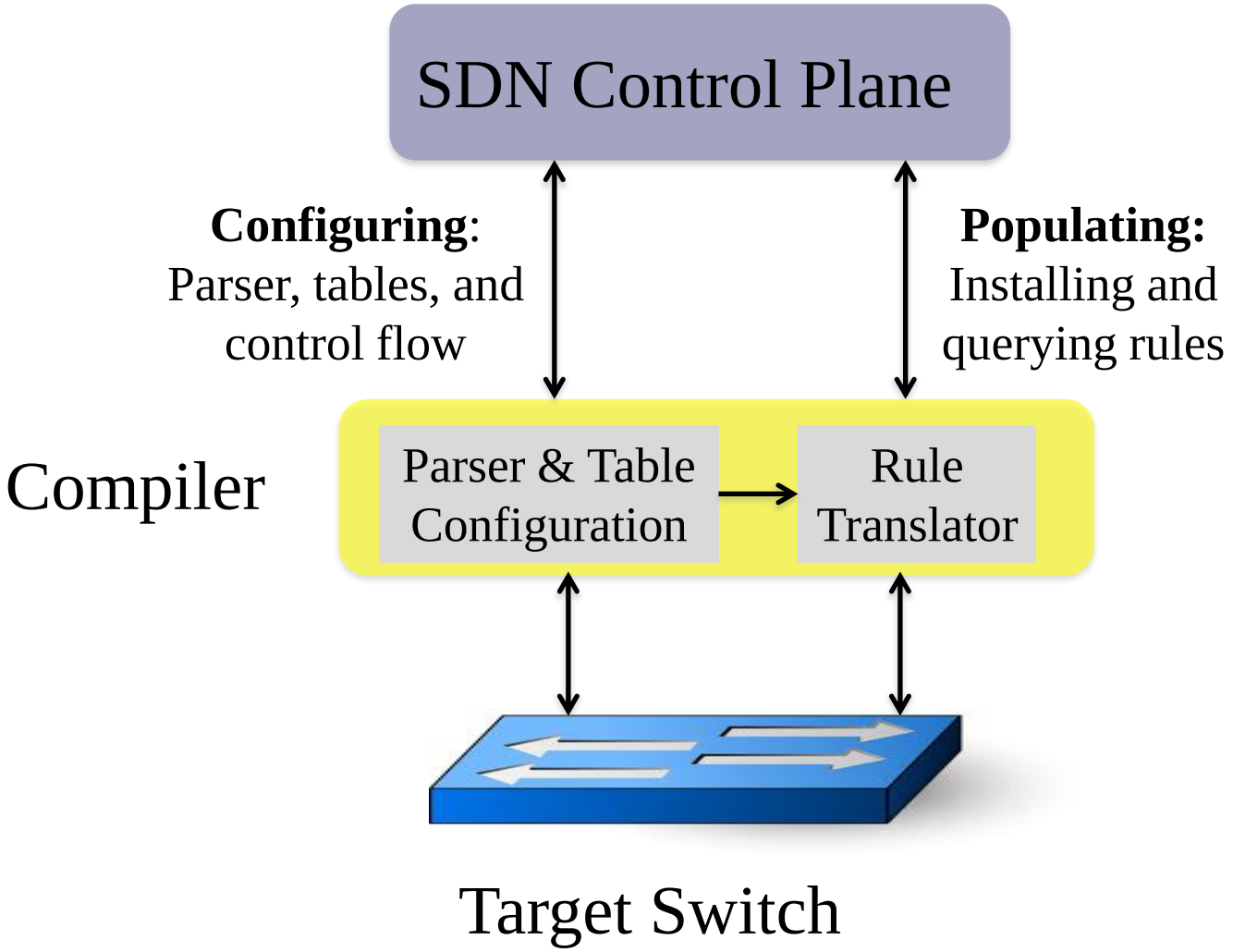
Software-defined networking (SDN) took a key step in abstracting network functions by separating the roles of the control and forwarding planes via an *open* interface between them (e.g., OpenFlow [27]). The control plane is lifted up and out of the switch, placing it in external software. This programmatic control of the forwarding plane allows network owners to add new functionality to their network, while replicating the behavior of existing protocols. OpenFlow has become quite well-known as an interface between the control plane and the forwarding plane based on the approach known as "Match-Action". Roughly, a subset of packet bytes are matched against a table; the matched entry specifies a corresponding action(s) that are applied to the packet.

One can imagine implementing Match-Action in software on a general purpose CPU. But for the speeds we are in-

# “Classic” OpenFlow (1.x)

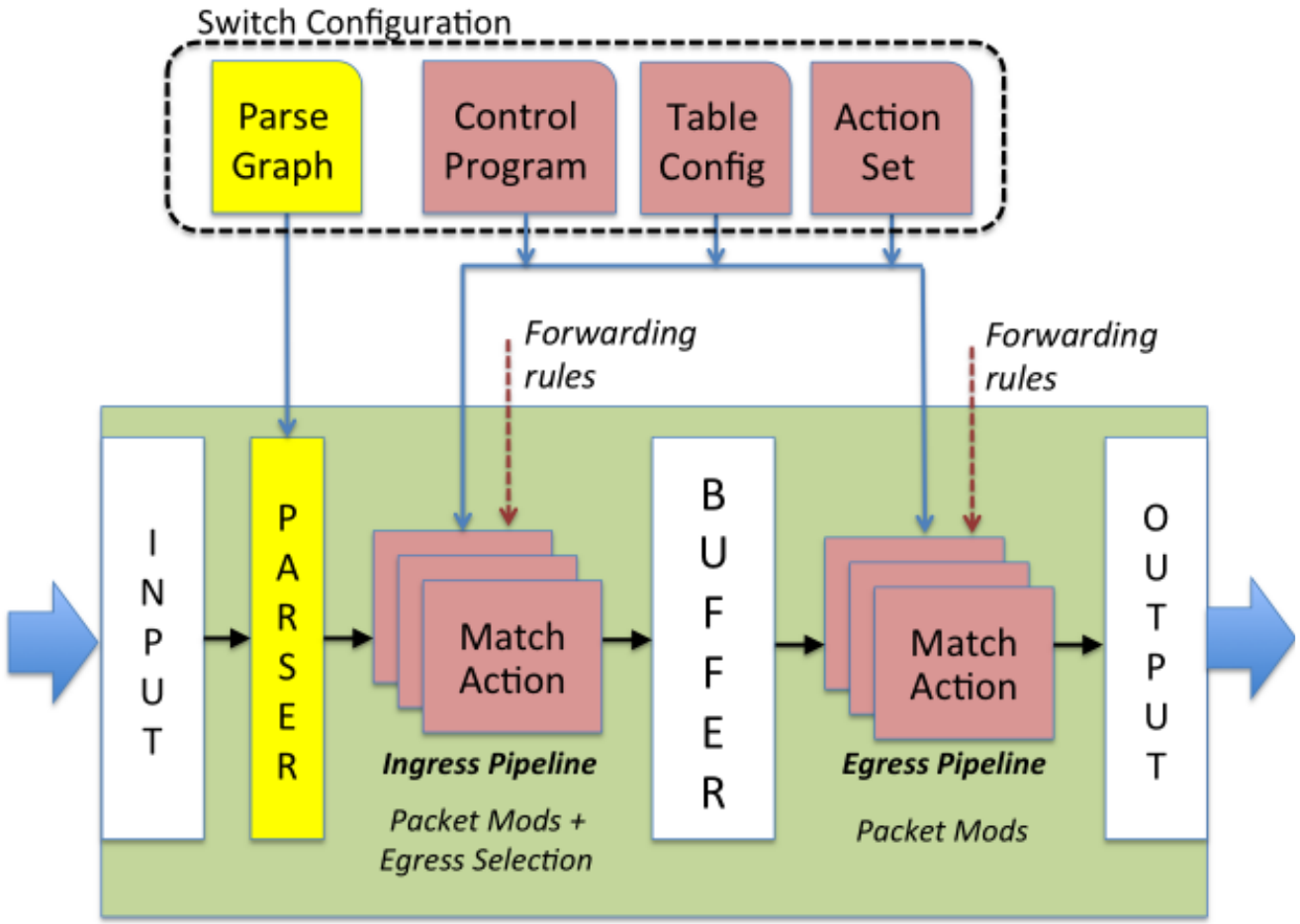


# “OpenFlow 2.0”

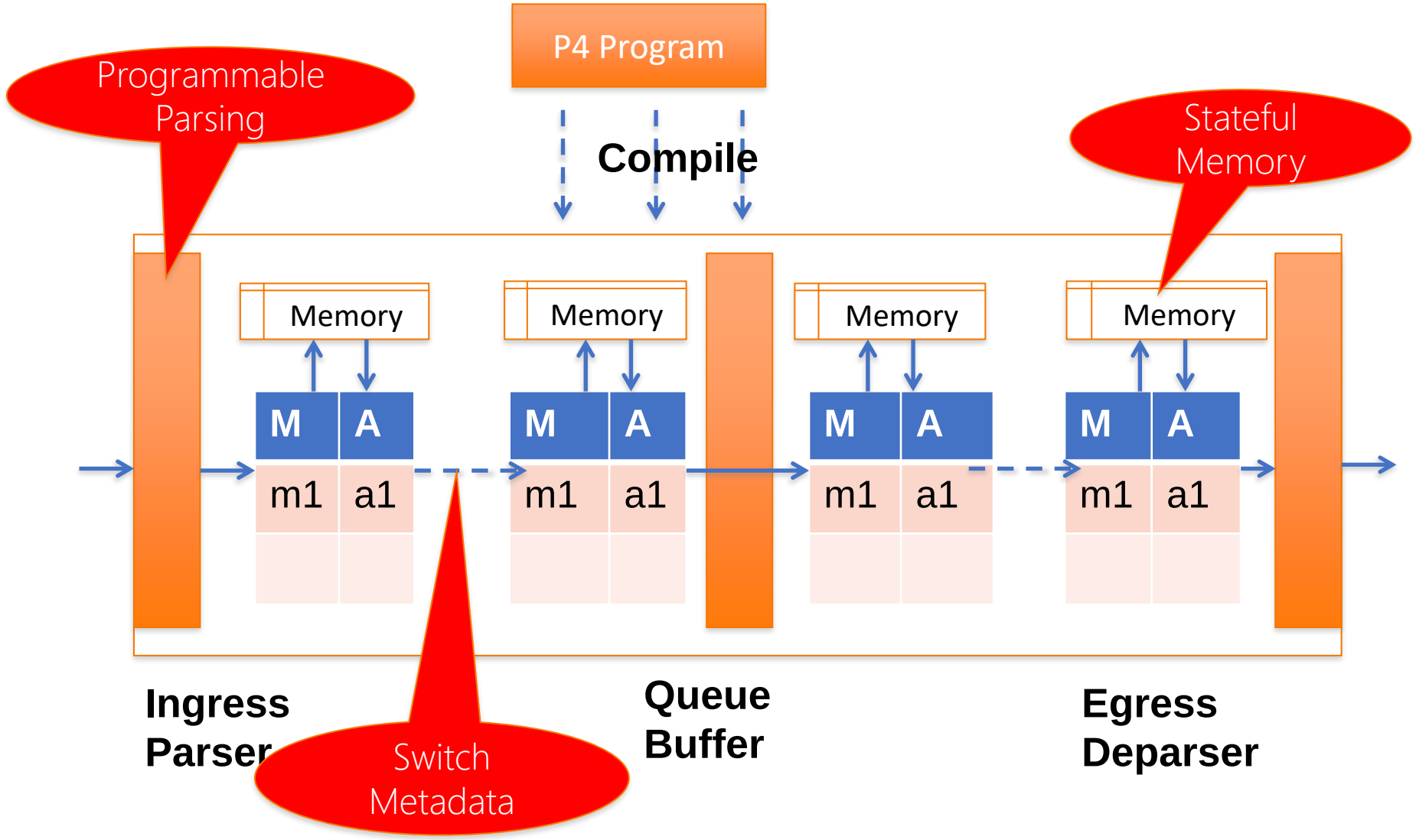

SDN Control Plane
Configuring:
Parser, tables, and
control flow
Populating:
Installing and
querying rules
Compiler
Parser & Table
Configuration
Rule
Translator
Target Switch

# P4 Abstract Model

- Reconfigurable parser
- + Non-sequential match-action
- + Protocol independent processing



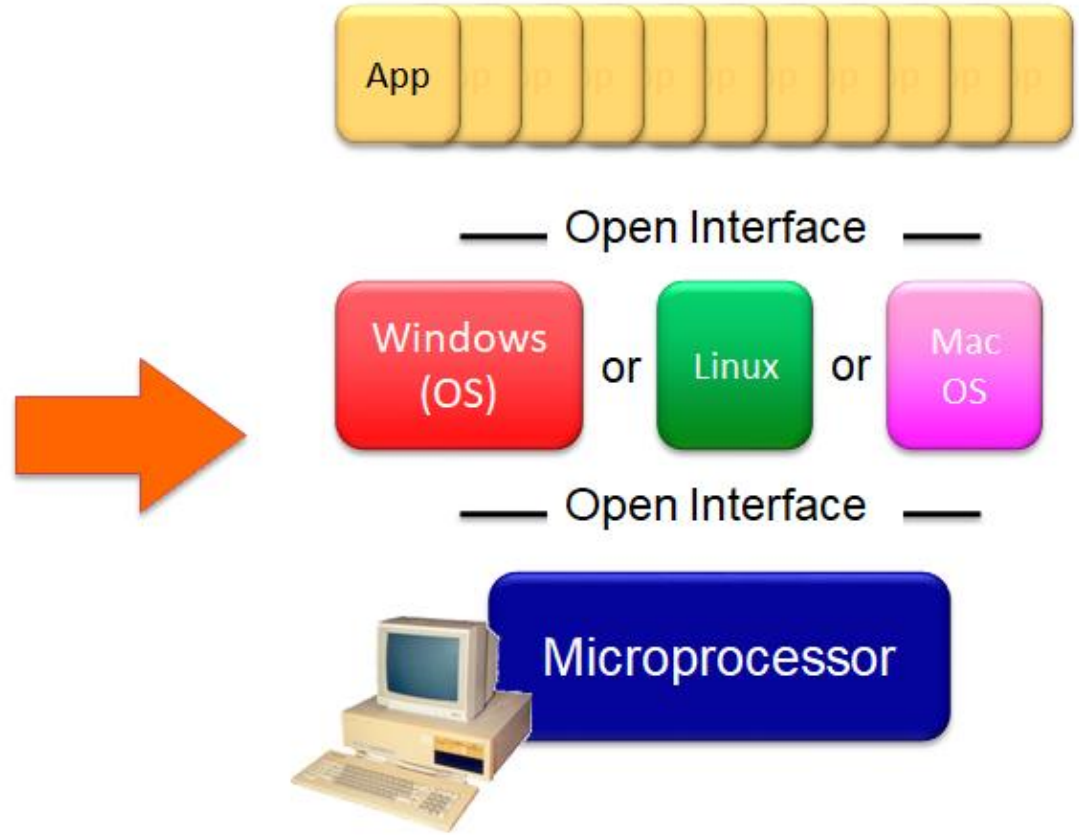
# Programmable Switches: Capabilities



# Analogy



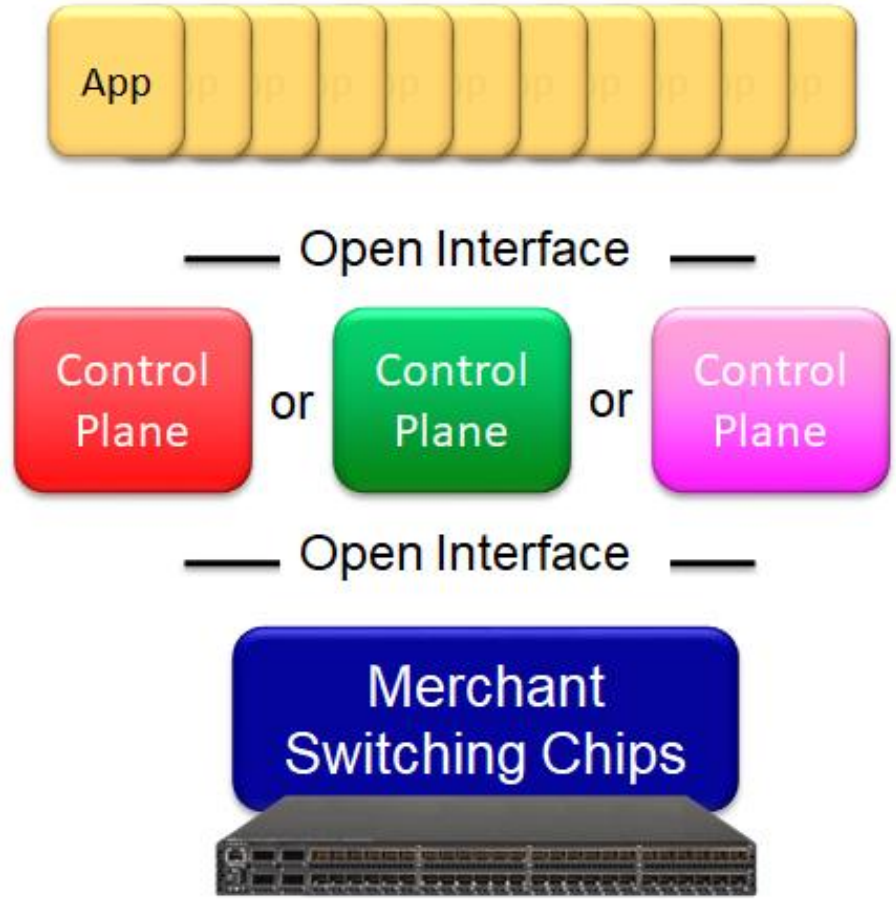

Vertically integrated
Closed, proprietary
Slow innovation
Small industry



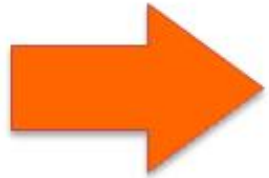

Horizontal
Open interfaces
Rapid innovation
Huge industry






Vertically integrated
Closed, proprietary
Slow innovation



Horizontal
Open interfaces
Rapid innovation



University of Colorado **Boulder**

# Google's Andromeda

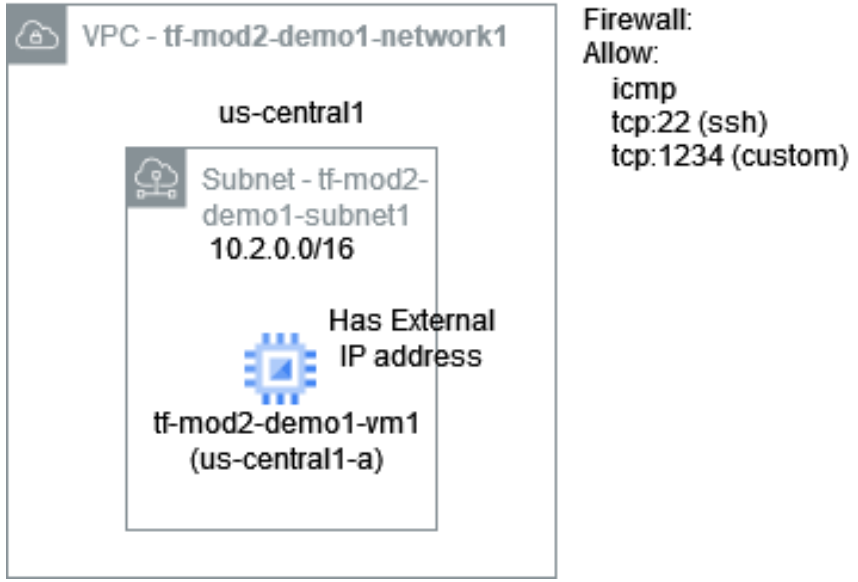
Course: Networking Principles in Practice – Cloud Networking
Module: VPC



University of Colorado **Boulder**

# Network Virtualization

- We saw we can create VPCs, subnets, VMs.
- We saw internal/external IP addresses
- These VMs can talk to other VMs, with a policy
- The VMs need to be isolated from other tenants (users of the cloud)
- How does google do this?



# Switch to their slides



University of Colorado **Boulder**

# Azure’s AccelNet (SmartNIC)

Course: Networking Principles in Practice – Cloud Networking
Module: VPC



University of Colorado **Boulder**

# Azure also uses a virtual switch

- But, this describes how they went one step further to improve performance.

# Switch to their slides



University of Colorado **Boulder**

# VPC – a bit deeper
## (and programming assignment overview)

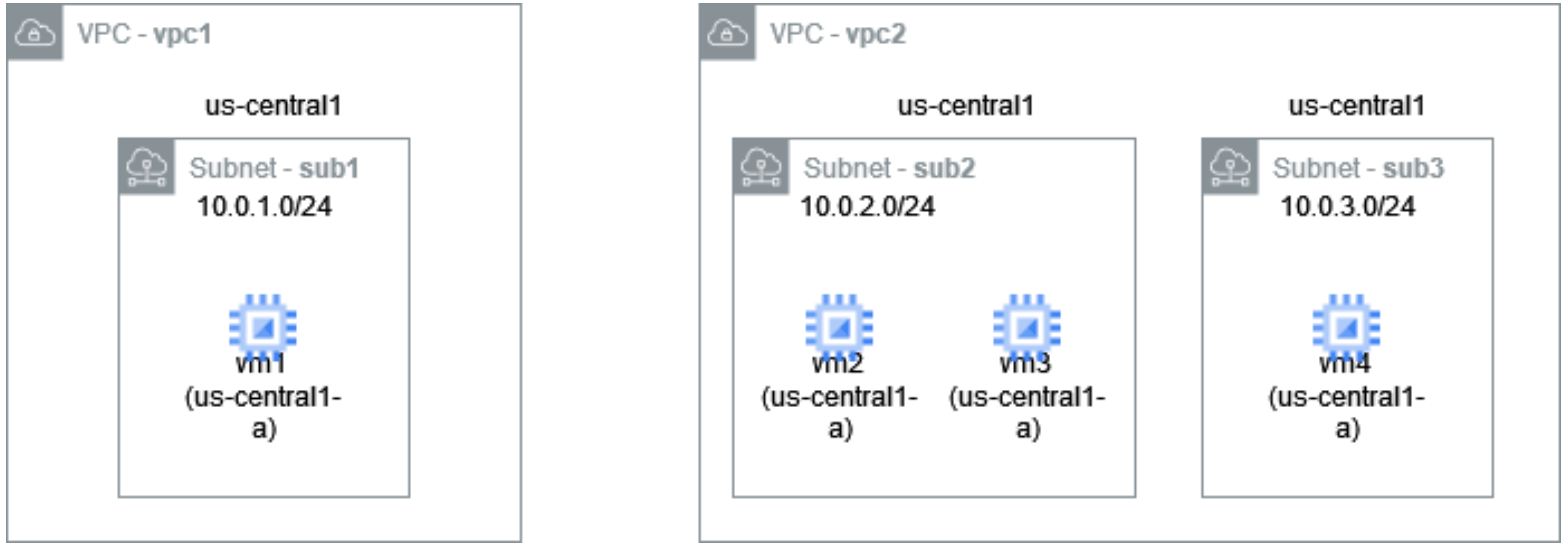
Course: Networking Principles in Practice – Cloud Networking
Module: VPC



University of Colorado **Boulder**

# Walkthrough 2 in GCP Console
# (note: programming assignment is this in terraform)

Create the following VPCs, Subnets, VMs in GCP Console
Give vm1 and vm2 external IP addresses
Allow icmp, tcp:22, and tcp:1234
Test connectivity (using nc) between:
- vm1 and vm2 (via public/ext IP)
- vm2 and vm4 (via private/int IP)



# Programming Assignment for Module 2 is doing this in terraform

- Use the demo code as a starter: https://github.com/eric-keller/npp-cloud/tree/main/tf-mod2-demo1

# Assessment

- Create your terraform code, and when you believe it is correct
- Run: terraform apply
- Run: terraform show --json |jq > terraform_show_out.json
- Perform some manual connectivity tests, and jq queries.  Noting output.
- Then answer some questions on coursera.

In development (so can try it out) – performs some checks:

python3 cloud_extract.py mod2 terraform_show_out.json

https://github.com/eric-keller/npp-cloud/tree/main/cloud-extract



University of Colorado **Boulder**